2026-5-24 C++继承：上

- 1. 继承的概念以及定义
  - 1.1 继承的概念
  - 1.2 继承的定义
    - 1.2.1 定义格式
    - 1.2.2继承基类成员访问方式的变化
  - 1.3 继承类模板
- 2. 基类和派生类间的转换
- 3.继承中的作用域
  - 3.1 隐藏规则
  - 3.2 考察继承作用域相关的选择题
    - 3.2.1 A和B中两个func构成什么关系？（）
    - 3.2.1 下面程序的编译运行结果是什么？（）

1. 继承的概念以及定义

1.1 继承的概念

继承（inheritance)机制是面向对象程序设计使代码可以复用的重要手段，它允许我们在保持原有类特性的基础上进行扩展，增加方法（成员函数）和属性（成员变量），这样产生新的类，称派生类。继承呈现了面对象程序设计的层次结构，体现了由简单到复杂的认知过程。以前我们接触的函数层次的复用，继承设计是类设计层次的复用。

```
//将公有成员都放到Person类中
class Person {
public:
    //进入校园/图书馆/实验室刷二维码等身份认证
    void identify() {
        //......
    }
protected:
    string _name = "张三";//姓名
    string _address;      //地址
    string _tel;          //电话
    int _age = 18;        //年龄
};
class Student :public Person//Student类继承自Person
{
public:
```

```
    //学习
    void study() {

    }
protected:
    int _stuid;//学号
};
class Teacher :public Person//Student类继承自Person
{
public:
    //授课
    void teacher() {

    }
protected:
    string title;//职称
};
int main() {
    Student s;
    Teacher t;
    s.identify();
    t.identify();
}
```

在上述代码中，Teacher和Student都具有Person类的特征，那么我们就可以让Teacher和Student都继承自Person，再在各自类中写入各自的特性。这样对Person中的成员进行一个复用，又保证了不同类的特点，避免了重复定义。

1.2 继承的定义

1.2.1 定义格式

举个例子：下面的Person是基类，也称作父类。Student是派生类，也称作子类。（基类/派生类，父类/子类）。

```
class Teacher :public Person
{        派生类     继承方式     基类
public:
    void teacher() {

    }
protected:
    string title;//职称
};
```

继承方式对照👇：



1.2.2继承基类成员访问方式的变化

1. **基类private成员在派生类中无论以什么样的方式继承都是不可见的**。这里的不可见是指基类的私有成员还是被继承到了派生类对象中，但是语法上限制派生类对象不管类里面还是类外面都不能去访问它。
2. 基类private成员在派生类中是不能被访问，如果基类成员不想在类外直接被访问，但需要在派生类中能访问，就定义为protected。（这里可以看出保护成员限定符是因为继承才出现的）。

类成员/继承方式	public继承	protected继承	private继承
基类的public成员	派生类的public成员	派生类的protected成员	派生类的private成员
基类的protected成员	派生类的protected成员	派生类的protected成员	派生类的private成员
基类的private成员	在派生类中不可见	在派生类中不可见	在派生类中不可见

3. 我们总结一下上面的表格，基类的私有成员在派生类中都是不可见的。基类的其他成员在派生类中的访问方式就等于：min(成员在基类的访问限定符，继承方式)，其中：public>protected>private。
4. 使用关键字class时默认的继承方式是private,使用struct时默认的继承方式时public，最好显示的写出继承方式。
5. 在实际运用中一般使用的都是public继承，几乎很少使用protected/private继承，也不提倡使用protected/private继承，因为protected/private继承下来的成员都只能在派生类的类里面使用，实际中扩展维护性不强。
   简单演示一下：

```
//示例演示三种继承关系下基类成员的各类型成员访问关系的变化
class Person {
public:
    void Print() {
        cout << _name << endl;
    }
protected:
    string _name = "张三";//姓名
private:
    int _age;
};

//class Student:protected Person
//class Student:private Person
class Student :private Person
{
protected:
    int _stunum;//学号
};
```

1.3 继承类模板

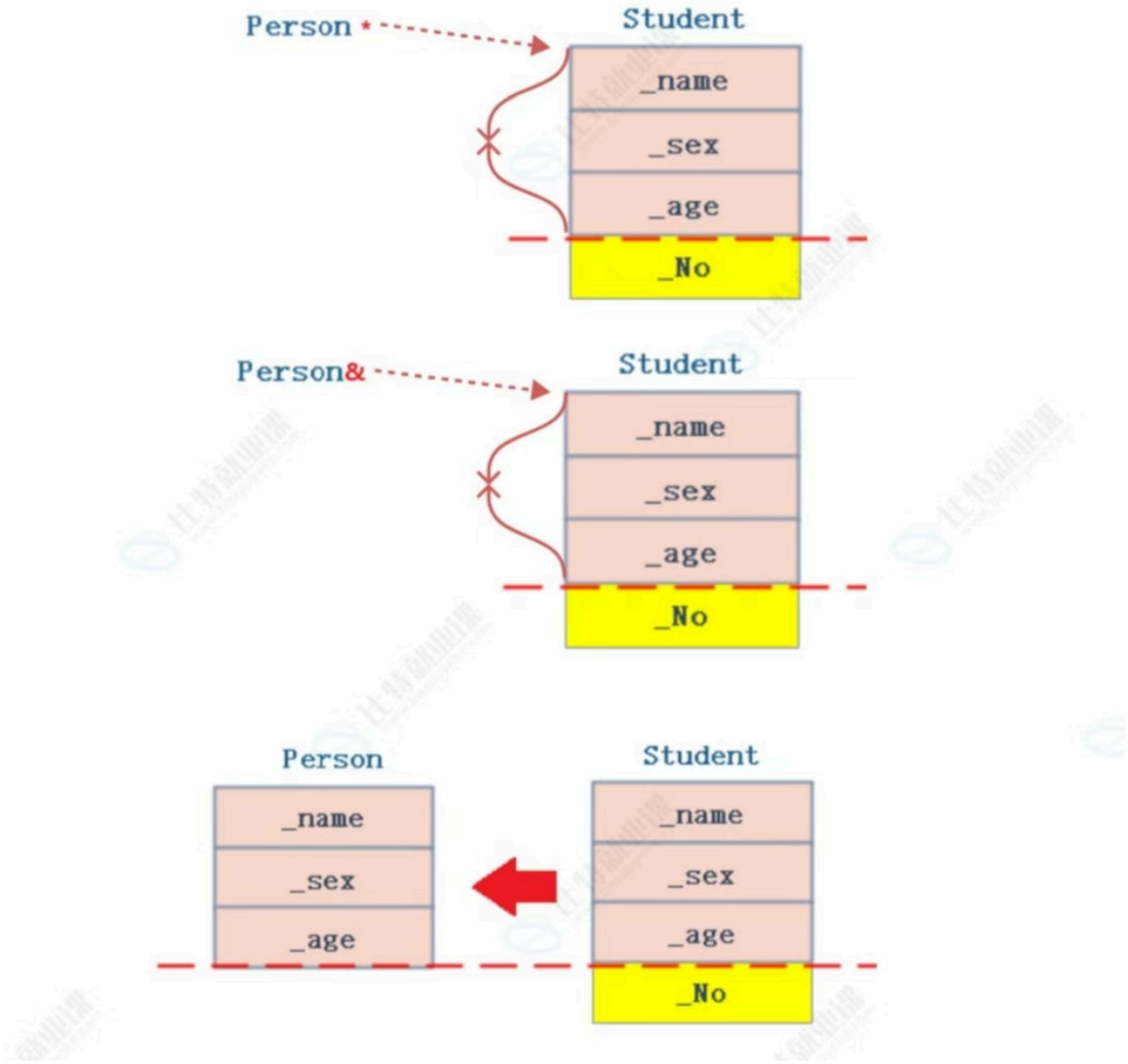
基类是类模板的时候，需要指定一下类域,否则编译会报错，找不到`"push_back"`标识符。原因是因为stack`<int>`实例化时，也实例化了`vector<int>`了。但是模板是按需实例化，`push_back`等成员函数未实例化，所以找不到`push_back`这个成员。

```
namespace ZL
{
    //例如stack继承自vector
    template<class T>
    class stack:public std::vector<T>
```

```
    {
    public:
        void push(const T& x) {
            vector<T>::push_back(x);
            //push_back(x);这样写是不对的
        }
        void pop() {
            vector<T>::pop_back();
        }
        const T& top() {
            return vector<T>::back();
        }
        bool empty() {
            return vector<T>::empty();
        }
    };
}
int main() {
    ZL::stack<int> st;
    st.push(1);
    st.push(2);
    st.push(3);
    while (!st.empty()) {
        cout << st.top() << " ";
        st.pop();
    }
    return 0;
}
```

2. 基类和派生类间的转换

1. 通常情况下我们把一个类型的对象赋值给另一个类型的指针或者引用时，存在类型转换，中间会产生临时对象，所以需要加const，如：`int a =1; const double& d = a;` public继承中，就是一个特殊处理的例外，派生类对象可以赋值给**基类的指针/基类的引用**，而不需要加const，这里的指针或者引用，可能指向基类对象，也可能指向派生类对象。
2. 派生类对象赋值给基类对象是通过基类的拷贝构造函数或者赋值重载函数完成的，这个过程就像派生类自己定义部分成员切掉一样，所以也被叫做切割或者切片。



- **指针与引用转换（图1、图2）：**
  - `Person*` 或 `Person&` 指向/引用 `Student` 对象时，仅仅是**视角的改变**。
  - **特点：** 底层的内存结构没有变化，`Student` 对象依然完整存在（包括 `_No`），只是通过基类指针或引用只能“看”到和访问基类部分（`_name`，`_sex`，`_age`），无法访问派生类特有的成员。这就像给 `Student` 戴上了一个 `Person` 的面具。
- **值拷贝转换（图3）：**
  - 将 `Student` 对象赋值给 `Person` 对象时，发生了**切片**。
  - **特点：** 这是一个**物理上的拷贝过程**。系统只复制了 `Student` 中属于 `Person` 的那部分数据（`_name`，`_sex`，`_age`）到一个全新的、独立的 `Person` 对象中。`Student` 特有的部分（`_No`）被直接丢弃（切掉），且两个对象在内存中是完全独立的个体。

3. 基类对象不能赋值给派生类对象,派生类对象可以通过拷贝构造赋值给基类对象。

```
//将公有成员都放到Person类中
class Person {
public:
    //进入校园/图书馆/实验室刷二维码等身份认证
    void identify() {
        //......
    }
protected:
    string _name = "张三";//姓名
    string _address;      //地址
    string _tel;          //电话
    int _age = 18;        //年龄
};
class Student :public Person//Student类继承自Person
{
public:
    //学习
    void study() {

    }
protected:
    int _stuid;//学号
};

int main() {
    Student sobj;
    //派生类对象可以直接赋值给基类的指针/引用
    Person* pp = &sobj;
    Person& rp = sobj;
    //派生类对象可以直接赋值给基类对象是通过调用拷贝构造完成的
    Person pobj = sobj;
    //基类对象不能直接赋值给派生类对象，否则编译会报错
    //sobj = pobj;
    return 0;
}
```

4. **基类的指针或者引用可以通过强制类型转换赋值给派生类的指针或者引用**。但必须是基类的指针是指向派生类对象时才是安全的。这里的基类如果是多态类型，可以使用RTTI（Run-Time Type Information)的dynamic_cast来进行识别后进行安全转换。

3.继承中的作用域

3.1 隐藏规则

1. 在继承体系中，基类和派生类都有独立的作用域。
2. **派生类和基类中有同名成员**，派生类成员将屏蔽基类对同名成员的直接访问，这种情况叫隐藏。（在派生类成员函数中，可以使用 基类::基类成员显示访问）

```
class Person {
public:
    //进入校园/图书馆/实验室刷二维码等身份认证
    void identify() {
        //......
    }
protected:
    string _name = "张三";//姓名
    string _num="222222222222";      //身份证号
};
class Student :public Person
{
public:
    //学习
    void Print() {
        cout << "姓名：" << _name << endl;
        //这里出现了同名的成员，默认都会调用派生类的成员
        // 如果不加类限定符号则基类的_num就会被隐藏
        cout << "身份证号码：" <<Person:: _num << endl;
        cout << "学号：" << _num << endl;
    }
protected:
    string _num = "11111";  //学号
};

int main() {
    Student s1;
    s1.Print();
    return 0;
}
```

上述代码的Student的_num和Person的_num构成隐藏关系，这样的代码很危险能运行但不正确。
3. 需要注意的是如果是成员函数的隐藏，只需要函数名相同就构成隐藏。
4. 注意在实际中继承体系里面最好不要定义同名的成员。

3.2 考察继承作用域相关的选择题

3.2.1 A和B中两个func构成什么关系？（）

A.重载 B.隐藏 C.重写 D.无关
只需要函数名相同就构成隐藏，哪怕参数不同。选择B。

3.2.1 下面程序的编译运行结果是什么？（）

A. 编译报错 B.运行报错 C.正常运行
`b.fun()`调用的是派生类的`fun()`没有参数编译就不通过，基类的`fun()`被隐藏了。
正确调用基类`fun()`:

```
b.A::fun();
```

题目：

```
class A {
public:
    void fun() {
        cout << "func()" << endl;
    }
};
class B :public A
{
public:
    void fun(int i) {
        cout << "func(int i)" << endl;
    }
};
int main() {
    B b;
    b.fun(10);
    b.fun();
    return 0;
}
```